BABEŞ-BOLYAI UNIVERSITY
FACULTY OF MATHEMATICS AND COMPUTER SCIENCE

Bachelor Degree Written Exam
Computer Science – English

VARIANT 2

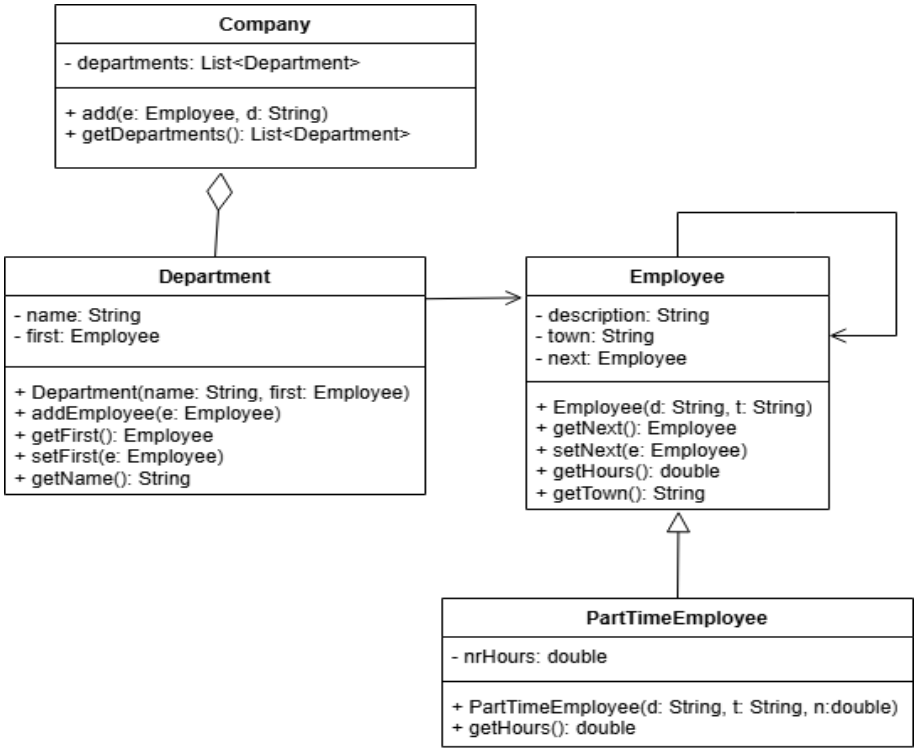
REMARKS

- All subjects are compulsory and full solutions are requested.
- The minimum passing grade is 5.00.
- The working time is 3 hours.

SUBJECT Algorithms and Programming

- **The programming language that is used must be indicated.**
- **The implementations are not required to deallocate dynamically allocated memory areas.**
- **Lack of appropriate programming style (suggestive variable names, indentation of code, comments, if necessary, readability of code) will result in a 10% deduction from the subject's score.**
- **Do not add additional attributes or methods, other than those mentioned in the statement, except for constructors and destructors, if applicable. Do not change the visibility of attributes specified in the statement.**
- **Do not use sorted containers, predefined sort and search operations.**
- **Existing libraries (from C++, Java, C#, Python) may be used for data types.**

1. **(2.5 points)** Define a function in one of the programming languages **Python, C++**, **Java** or **C#**, which receives as parameters an alphabetically ordered list *lst* of strings (representing department names in a company) and a string *name*, representing the name of a department. Assuming (as a precondition of the function) that the string *name* does not appear in the list *lst*, the function will add the string *name* to the list *lst*, such that the list *lst* remains alphabetically ordered by the names of the departments. A recursive implementation is required. Auxiliary functions may be implemented. Iterative solutions will be awarded partial score.
2. **(1 point)** Indicate the time complexity in the *best-case*, *average-case*, and *worst-case* scenarios for the function from item **1**. Justify your answer.
3. **(5.5 points)** Consider the following UML diagram, which contains the classes **Company**, **Department** (representing a collection of employees), **Employee** (full-time employee), **PartTimeEmployee**.



- The attributes *description* and *town* in class **Employee** (job description and town of origin) and *name* in class **Department** (department name) must be non-empty, and the attribute *nrHours* in class **PartTimeEmployee** (employee's monthly number of hours) must be a strictly positive number, smaller than 160. Constructors must enforce the constraints.
- The class **Employee** has a method **getHours()** that returns 160 and a method **getTown()** that returns the employee's town of origin. The **getNext()** and **setNext(e: Employee)** methods get and set the *next* attribute, respectively.
- The method **getHours**() in class **PartTimeEmployee** returns the part-time employee's monthly number of hours (the *nrHours* attribute in the class).
- The class **Department** stores a collection of employees (full-time or part-time) in the department. The **addEmployee(e: Employee)** method adds an employee to the department, and the **getName()** method returns the department's name. The **getFirst()** and **setFirst(e: Employee)** methods get and set the *first* attribute, respectively.
- The class **Company** stores a list of departments. The **add(e: Employee, d: String)** method adds employee *e* to the department with name *d* (if a department named *d* does not exist, it will be created). The **getDepartments()** method returns the list of departments in the company.

Write a program that implements the following requirements using one of the **C++, Java**, or **C#** programming languages:

a) Declare all classes, attributes, and methods as per the diagram above. Implement only the following methods:

a1) Method **setNext**(**e**: **Employee**) in class **Employee**.

a2) The constructors of classes **Department** and **PartTimeEmployee**.

a3) Methods **addEmployee(e: Employee)** and **setFirst (e: Employee)** in class **Department.**

For the method **addEmployee(e: Employee)** an implementation with a worst-case time complexity $\theta(1)$ is required. Solutions that do not meet the required complexity will be awarded partial score.

a4) Method **add(e: Employee, d: String)** in class **Company**.

b) Define a function that receives as a parameter an object *c* of type **Company**, calculates the maximum number of employees (*m*) having the same town of origin and then displays all towns from which *m* employees originate (each town will be displayed only once). Assuming that town names have at most 20 characters, an implementation with an average time complexity of $\theta(n)$ is required (*n* being the number of employees in the company). Solutions that do not meet the required complexity will be awarded partial score. **Using predefined functions in libraries to determine distinct elements in a sequence is not allowed.**

c) Create an object of type **Company** and add 4 employees, 2 of which are part-time employees (your choice). Then call the function from item **b)**.

SUBJECT Databases **VARIANT 2**

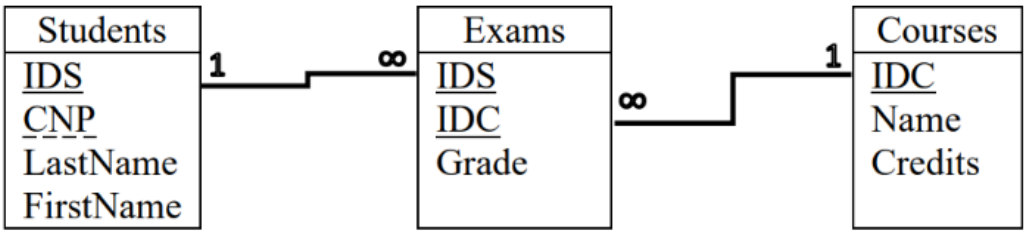
Problem 1. (4 points)

A software company develops applications for various industries using technologies tailored to each specific domain. To manage the allocation of employees from departments to the projects carried out within the company, a relational database is used:

- An employee has a code, a hiring date, a name (this can be a simple attribute, a string), a unique email address (unique for each employee), and a monthly salary; additionally, each employee is assigned to a single department and a single project.
- Each department has a code and a name; a department can handle multiple projects at the same time.
- A project has a code, a unique internal project name, a budget, and a dominant technology; for example, the project with internal name P1T1 has ".NET" as its dominant technology. A project is carried out by a single department.
- The company operates across several locations where employees can perform their activities: a location has a code, an address (can be a simple attribute, a string), and a phone number (unique for each location); a location may or may not have parking and may or may not have a cafeteria.
- Employees from multiple departments may work at a single location, but for each location it is predefined which departments operate there. A location can be allocated to multiple departments, and a department may be assigned to multiple locations.

Create a BCNF relational schema for the database, clearly indicating the primary keys, candidate keys, and foreign keys. Build the schema using one of the methods shown in the example below:
* example for the tables Students, Courses, and Exams:

V1. Diagram with tables, primary keys underlined with a solid line, candidate keys underlined with a dashed line, and links drawn directly between foreign keys and their corresponding primary/candidate keys (for example, a link drawn between the column CodS in Exams and the column CodS in Students).



V2.
Students[IDS, CNP, LastName, FirstName]
Courses[IDC, Name, Credits]
Exams[IDS, IDC, Grade]
Primary keys are underlined with a solid line, while candidate keys are underlined with a dashed line. {IDS} in Exams is a foreign key referencing {IDS} in Students. {IDC} in Exams is a foreign key referencing {IDC} in Courses.

Problem 2. (5 points)

The following relational schemas are given:

Authors[AuthorID, Name, Country, BirthYear]

Books[BookID, Title, Field, PageCount, *AuthorID*]

Publishers[PublisherID, Name]

BooksPublishers[*BookID*, *PublisherID*, Price, YearPublished]

Primary keys are underlined. Foreign keys are written in italics and have the same name as the columns they reference.

a. Write an SQL query that returns the code and name of each author who has written at least 2 books in a single year and has at least one book published by the publisher "DE2".

b. The following instances are given for the relations Books, Publishers, and BooksPublishers:

Books

BookID	Title	Field	PageCount	AuthorId
1	T1	D2	125	A1
2	T2	D1	175	A1
3	T3	D2	210	A3
4	T4	D3	98	A2
5	T5	D2	113	A4

Publishers

PublisherID	Name
E1	DE1
E2	DE2
E3	DE3

BooksPublishers

BookID	PublisherID	Price	YearPublished
1	E2	34	2021
2	E1	28	2022
3	E3	55	2021
4	E2	21	2024
5	E1	42	2023

b1. Specify the result of evaluating the query below on the given instances. Indicate only the values of the tuple/tuples and the names of the columns in the result, without presenting all the steps of the query evaluation.

```
SELECT t1.AuthorID, t1.Title, t1.BookID
FROM Books t1
WHERE
EXISTS
        (SELECT BookID FROM BooksPublishers t2
        INNER JOIN
        (SELECT ce.PublisherID FROM BooksPublishers ce
                GROUP BY ce.PublisherID
                HAVING COUNT(*)>1 AND SUM(ce.Price)<60
        UNION ALL
        SELECT e.PublisherID FROM Publishers e WHERE e.Name='DE2'
        )t3 ON t3.PublisherID=t2.PublisherID
                WHERE t2.BookID=t1.BookID
        )
```

b2. Explain whether the following functional dependencies are satisfied or not by the data in the Books instance.

- {PageCount} → {BookID}
- {AuthorID} → {Field}

VARIANT 2

SUBJECT: Operating Systems

Problem 1 (5 points). Answer the following questions about the execution of the program `./a` below, executed initially without command line arguments, considering that all necessary includes are present and that the `fork` system call and at least one of the `exec` system calls execute successfully.

```
 1  int main(int argc, char** argv) {
 2      if(argc > 1) {
 3          printf("%s\n", argv[1]);
 4      }
 5      fork();
 6      execlp("a", "a", "y", NULL);
 7      execl("./a", "./a", "z", NULL);
 8      printf("%d\n", wait(NULL));
 9      return 0;
10  }
```

a) Explain in detail the functioning of line 6.

b) How many times will the value of `argv[1]` be displayed in the console? Justify your answer.

c) What will be displayed in the console if the `PATH` environment variable does not include the current directory?

d) How many processes will execute the `wait` system call?

e) What will be displayed in the console if the `PATH` environment variable includes the current directory and line 5 is removed?

Problem 2 (4 points). Answer the following questions about the execution of the UNIX Shell script below, considering that the file `a.txt` exists and contains on each line sequences of letters, separated by spaces.

```
 1  #!/bin/bash
 2
 3  while read L; do
 4    R=""
 5    for W in $L; do
 6      X=""
 7      C=`echo $W | sed -E "s/(.)/\1 /g"`
 8      for A in $C; do
 9        X="$A$X"
10      done
11      R="$X $R"
12    done
13    echo $R
14  done < a.txt
```

a) What values will variables L and W take?

b) Explain in detail the functioning of line 7.

c) What will display in the console the execution of the script if file `a.txt` has the following content?

```
abc def
pqr xyz
```

d) What will display in the console the execution of the script, for file `a.txt` defined at point c), if in line 7 the space after the digit 1 is removed?

BAREM INFORMATICĂ
VARIANTA 2

Subiect Algoritmică și Programare

Oficiu – **1p**

Cerința **1.** **– 2.5p**

- signatura – **0.2p**
- implementare recursivă – **2.3p** din care
  - adăugare lista vidă – 0.5
  - adăugare la începutul/sfârșitul listei (pentru caz favorabil) – 0.5
  - apelul recursiv – 1.3

* implementare iterativă corectă – **0.7**p

Cerința **2.** **– 1p**

- caz favorabil **0.2p** din care
  - complexitate $\theta(1)$ – 0.1
  - justificare – 0.1
- caz mediu **0.4p** din care
  - complexitate $\theta(n)$ – 0.2 (n – dimensiunea listei *lst*)
  - justificare – 0.2
- caz defavorabil **0.4p** din care
  - complexitate $\theta(n)$ – 0.2 (n – dimensiunea listei *lst*)
  - justificare – 0.2

Cerința **3.a)** **– 2.6p**

Definirea clasei Employee – **0.6p** din care
- atribute – 0.1
- contructor – 0.1
- metoda **setNext (a1)** – 0.2
- metode **getNext**, **getHours**, **getTown** – 0.2

Definirea clasei PartTimeEmployee– **0.55p** din care
- relația de moștenire – 0.2
- atribut – 0.05
- **constructor (a2)** – 0.2
- metoda **getHours** – 0.1

Definirea clasei Department– **1p** din care
- atribute – 0.1
- **constructor (a2)**– 0.1
- metoda **addEmployee (a3)** – 0.5
  - * punctajul se acordă pentru implementare având complexitate $\theta(1)$
  - * pentru implementari având complexitate mai mare decat $\theta(1)$ se acorda 0.25p
- metoda **setFirst (a3) –** 0.2
- metode **getFirst**, **getName –** 0.1

Definirea clasei Company – **0.45p** din care
- atribut, metoda **getDepartments** – 0.1p
- metoda **add (a4)** – 0.35p

Funcția **3.b)** – **2.6p**

- signatura – **0.1p**
- implementare având complexitatea timp în caz mediu $\theta(n)$ – **2.5p**
  - * implementare având complexitatea timp în caz mediu $\theta(n\log_2 n)$ – **1.5p**
  - * implementare având complexitatea timp în caz mediu $\theta(n^2)$ – **0.75p**

Funcția principală **3.c)** – **0.3p**

- construire obiect de tip Company – 0.1p
- adăugare angajați - 0.1p
- apel funcție b) pentru expresie– 0.1p

BAREM INFORMATICĂ
VARIANTA 2

Subiect Baze de date

Oficiu – **1p**

Problema 1. Punctaj - 4p

- relații cu atribute corecte, chei primare, chei candidat: **3p**
- legături modelate corect (chei externe): **1p**

Problema 2. Punctaj - 5p

- **a** - rezolvarea completă a interogării: **2.5p**

- **b1** - rezultat evaluare interogare:

CodAutor	Titlu	CodCarte
A1	T1	1
A2	T4	4

  - coloane – **0.5p**
  - valori tuplu – **1p**

- **b2** - {NrPagini}→{CodCarte} este satisfăcută – **0.25p**; **0.25p** explicație
  - {CodAutor}→{Domeniu} nu este satisfăcută – **0.25p**; **0.25p** explicație

Notă: La specializările Informatică engleză și Informatică maghiară se iau în considerare versiunile traduse în limbile corespunzătoare.

VARIANTA 2

SUBIECT Sisteme de operare

Barem:

1p – oficiu

Problema 1 (5p)

1p – a) Execută programul `a` cu argumentul `y`, dacă locaţia programului `a` se găseşte in `PATH`

1p – b) Se va afişa de o infinitate de ori, pentru că programul se reexecută la linia 6 sau 7.

1p – c) Se vor afişa o infinitate de caractere `z`

1p – d) Nicunul pentru că unul dintre cele două apeluri sistem `execl` şi `execlp` se vor executa cu succes şi vor suprascrie codul procesului înainte să se ajungă la linia 8.

1p – e) Se vor afişa o infinitate de caractere `y`

Problema 2 (4p)

1p – a) L va lua ca valori fiecare linie din fişierul a.txt, variabila W va lua ca valori fiecare secvenţe de litere din linia stocată în L

1p – b) După fiecare literă din variabila W se adaugă un spaţiu, rezultatul stocându-se in variabila C

1p – c) Se va afişa

```
fed cba
zyx rqp
```

1p – d) Se va afişa

```
def abc
xyz pqr
```